WHY FORBID-OVERRIDES-PERMIT SEMANTICS MATTER

One of the most important and sometimes surprising aspects of Cedar's evaluation model is that a `forbid` always overrides a `permit`. This rule is essential in a layered policy architecture, where foundational, shared operational, and application policies are assessed together. Without a clear and predictable override rule, combining policies from multiple stores could quickly lead to ambiguous or inconsistent results.

Consider what happens when an application policy permits an action, but a foundational policy enforces a security requirement that the request does not satisfy. If the application's permit could override the foundational restriction, the entire zero-trust model would fall apart. By prioritizing forbid rules, the policy engine guarantees that security-critical constraints stay in place, no matter how many application-level permissions are granted.

This behavior also enhances authorization safety during failures. If required context, such as device posture or authentication freshness, is missing or outdated, a foundational policy that relies on that context issues a forbid, causing the system to deny access. In a zero-trust architecture, this *fail-closed* behavior is a feature, not a flaw.

Practically speaking, forbid-overrides-permit means:

- Foundational security rules, written as `forbid` policies, are always enforced, even if application logic would otherwise allow an action.
- Shared operational rules are consistently applied across systems without depending on developers to add extra checks.
- Application policies remain focused on business logic, trusting foundational and operational layers to manage broader concerns.

A layered policy system only works when the evaluation model is predictable. Forbid-overrides-permit semantics provides that predictability, ensuring that combining policies from multiple stores results in a single, consistent, and secure decision every time.